

Voxel Terrain Demystified

How Modern Games Create Smooth, Fully Destructible Worlds

Taha Tahiri

June 18, 2026

Abstract

From realistic snow physics to fully destructible game environment, players are often offered the freedom to reshape the environment in real time while keeping the world smooth and responsive. At first glance, this seems incompatible with traditional polygon meshes, which are expensive to edit topologically and difficult to regenerate continuously.

This document explains the ideas behind such systems: scalar density fields, voxel-style storage, surface extraction by interpolation, incremental chunk rebuilding, spatial collision against a deformable world, and graph-like reasoning for connected terrain regions. The attached JavaScript implementation is used throughout as a concrete example, showing how these ideas become practical in a playable WebGL prototype.

1 The Problem

Suppose we want a player to punch a hole through a hill, tunnel sideways into it, and then pull out a chunk of the terrain as a separate object. With a traditional static mesh, that would be difficult because the geometry is not just being moved; it is being changed. New holes must be cut, new edges created, normals updated, and the scene must remain visually smooth.

A static mesh pipeline is good at drawing fixed surfaces, but less suited for large-scale destruction. The problem is not merely rendering; it is the fact that deformation changes the underlying topology. Once a surface is broken, the model must be rebuilt locally and the physics system must understand the new shape.

This is why destructible terrain systems usually do not store the world directly as triangles. Instead, they store a field describing how solid each region of space is, and they regenerate the visible surface when that field changes.

2 Thinking in Voxels

Most people imagine terrain as triangles, but many destructible terrain systems begin with voxels. A voxel is a volumetric sample in 3D space, analogous to a pixel in 2D. Instead of asking what color is at a point, we ask how much matter exists there.

In practice, the world is divided into a regular lattice:

[$V(i,j,k)$]

where each sample stores information about the terrain at that location. The important idea is that the voxel grid is usually not the final thing the player sees. It is an internal representation that drives editing, collision, and mesh generation.

In the attached JavaScript implementation, the world is not stored as a visible grid of blocks. The code keeps a map of density values and evaluates them on demand:

```
const densityMap = new Map();
function pKey(ix,iy,iz){ return ix + '|' + iy + '|' + iz; }
```

```
function getDensityPoint(ix,iy,iz){
const k = pKey(ix,iy,iz);
if(densityMap.has(k)) return densityMap.get(k);
const v = baseDensity(ix,iy,iz);
densityMap.set(k, v);
return v;
}
function setDensityPoint(ix,iy,iz,v){ densityMap.set(pKey(ix,iy,iz), v); }
function addDensityPoint(ix,iy,iz,delta){ setDensityPoint(ix,iy,iz, getDensityPoint(ix
,iy,iz) + delta); }
```

This pattern matters because it separates storage from appearance. The game can edit values locally without rebuilding the whole world, and the renderer can later query those values to decide where the surface exists.

3 The Scalar Field

A more powerful representation than a binary voxel grid is a density field. Instead of storing only ‘solid’ or ‘air,’ we store a continuous function:

[$f(x,y,z)$]
 where:

- $f(x,y,z) < 0$ represents solid terrain,
- $f(x,y,z) > 0$ represents empty space,
- $f(x,y,z) = 0$ defines the surface.

This is often called an implicit surface. The visible world is not stored directly. Instead, the surface is the set of points where the density crosses zero.

A simple terrain field can be written as:

[$f(x,y,z)=y-h(x,z)$]

where $h(x,z)$ is a height function. Points below the height are inside the ground; points above it are empty. Real games usually go beyond a single height function and add layered noise, caves, overhangs, and local edits.

The attached JavaScript uses exactly this style of field construction. The terrain starts with a base height and then gets modulated by 3D noise:

```
function terrainHeight(x,z){
return 18 + fbm3(x, 0, z) * 6 + fbm3(x*1.8, 0.3, z*1.8) * 3;
}
function baseDensity(x,y,z){
let d = y - terrainHeight(x,z);
d -= fbm3(x*0.85, y*0.85, z*0.85) * 1.3;
d -= fbm3(x*0.16, y*0.16, z*0.16) * 2.0;
d += Math.max(0, fbm3(x*0.22 + 18, y*0.22 - 11, z*0.22 + 7) - 0.12) * 4.5;
return d;
}
```

This is much richer than a flat heightmap. The first term gives a general hill shape, the next terms carve variation at different frequencies, and the final term creates occasional positive features that can become overhangs or denser formations. In other words, the terrain becomes a sculpted mathematical field rather than a fixed mesh.

4 Why the Terrain Looks Smooth

If the world is stored as voxels or density samples, why does it not look blocky? The reason is that the visible mesh is generated from the scalar field after the fact. The samples act as an invisible simulation layer, and a surface extraction algorithm reconstructs triangles that approximate the zero level set.

The attached implementation does this by decomposing each cube into tetrahedra and interpolating the intersection points on edges that cross the density threshold. This approach is closely related to marching cubes, but the tetrahedral decomposition reduces ambiguity and makes triangulation simpler to handle.

4.1 Surface Extraction by Interpolation

Inside a cube, each corner is classified as inside or outside according to the sign of the density. Wherever an edge connects a negative sample to a positive sample, the surface must cross that edge. The crossing point can be found by linear interpolation.

The code does this with the helper:

```
function interp(p1,p2,v1,v2){
const t = v1 === v2 ? 0.5 : (0 - v1) / (v2 - v1);
return [lerp(p1[0],p2[0],t), lerp(p1[1],p2[1],t), lerp(p1[2],p2[2],t)];
}
```

This is the key geometric step. Rather than drawing hard block faces, the renderer finds the exact transition point between solid and empty space. Because the interpolation is continuous, the resulting surface is smooth.

4.2 Normal Estimation

A smooth mesh also needs smooth lighting. For that, the gradient of the density field can be used as a normal vector. The gradient points in the direction of greatest increase, which is exactly what a surface normal needs for shading.

The implementation estimates the gradient numerically:

```
function gradientAt(x,y,z){
const e = 0.35;
const dx = sampleDensity(x+e,y,z) - sampleDensity(x-e,y,z);
const dy = sampleDensity(x,y+e,z) - sampleDensity(x,y-e,z);
const dz = sampleDensity(x,y,z+e) - sampleDensity(x,y,z-e);
return norm3(dx,dy,dz);
}
```

This gives the lighting system a normal vector even though the surface is implicit rather than explicitly stored. In practice, this is one of the reasons deformable terrain can still look polished and organic.

5 Local Deformation and Player Attacks

Now suppose the player throws a punch. The system should not delete a chunk of geometry outright. Instead, it should modify the density field locally and let the surface extraction step rebuild only the affected part of the world.

A common strategy is to apply a spherical or capsule-shaped edit. The basic idea is to add or subtract density in a neighborhood around the impact point. If the density is lowered enough, solid terrain becomes empty space and a crater appears.

The attached code implements spherical falloff in a way that keeps edits smooth:

```
function deformSphere(center, radius, amount){
const [cx,cy,cz] = center;
const minx = Math.floor(cx - radius - 1), maxx = Math.ceil(cx + radius + 1);
const miny = Math.floor(cy - radius - 1), maxy = Math.ceil(cy + radius + 1);
const minz = Math.floor(cz - radius - 1), maxz = Math.ceil(cz + radius + 1);
for(let x=minx; x<=maxx; x++) for(let y=miny; y<=maxy; y++) for(let z=minz; z<=maxz; z
++){
const dx = x - cx, dy = y - cy, dz = z - cz;
const d = Math.hypot(dx,dy,dz);
if(d > radius) continue;
const t = 1 - d / radius;
const fall = t*t*(3-2*t);
const delta = amount * fall * fall;
if(delta !== 0){
addDensityPoint(x,y,z, delta);
markPointDirty(x,y,z);
}
}
}
```

This function is important because it shows how the destruction effect is not a special visual trick. It directly edits the world representation. The crater shape comes from the falloff curve, which fades smoothly toward the edge of the impact radius. That smoothness prevents harsh seams and gives the terrain a more natural appearance.

The player-specific attacks are built on top of this same idea:

```
function handlePunch(kind){
if(attackCooldown > 0) return;
attackCooldown = 0.18;
const fwd = dirFromYawPitch(player.yaw, player.pitch);
const hand = [player.pos[0] + fwd[0]*1.5, player.pos[1] + 1.0 + fwd[1]*0.5, player.pos
[2] + fwd[2]*1.5];
if(kind === 'forward'){
deformCapsule([hand[0]+fwd[0]*0.6, hand[1]+fwd[1]*0.6, hand[2]+fwd[2]*0.6], [hand[0]+
fwd[0]*4.5, hand[1]+fwd[1]*4.5, hand[2]+fwd[2]*4.5], 1.7, 5.2, 5);
} else if(kind === 'up'){
const a = [player.pos[0] + fwd[0]*1.2, player.pos[1] + 1.0, player.pos[2] + fwd
[2]*1.2];
const b = [a[0] + fwd[0]*2.4, a[1] + 4.2, a[2] + fwd[2]*2.4];
deformCapsule(a, b, 1.5, 4.4, 5);
} else if(kind === 'down'){
const a = [player.pos[0], player.pos[1] - 0.2, player.pos[2]];
const b = [player.pos[0] + fwd[0]*0.8, player.pos[1] - 4.6, player.pos[2] + fwd
[2]*0.8];
deformCapsule(a, b, 1.4, 6.0, 6);
}
}
```

This matches the gameplay idea very well. A forward punch digs ahead, an upward hit sculpts vertical terrain, and a downward attack can dig into the ground. The attack is not just a combat action; it is a localized terrain editor.

6 Where Graph Theory Appears

One of the least obvious parts of destructible terrain is graph theory. Once the world is discretized into samples, the terrain naturally becomes a graph. Each voxel or density sample can

be treated as a node, and neighboring samples are connected by edges.

That means classical algorithms become useful immediately.

6.1 Connected Components

If the player destroys a support column, a rock overhang may become disconnected from the main landmass. The system then needs to determine whether that region should remain attached to the world or become a separate object.

This is a connected-components problem. Breadth-First Search and Depth-First Search can identify all samples reachable from a seed point. If a region is no longer connected to the terrain anchor, it can be detached and simulated independently.

6.2 Flood Fill and Region Extraction

Flood fill is useful for cave detection, enclosed spaces, and the extraction of disconnected terrain fragments. It is conceptually simple but powerful: starting from a known cell, visit all neighboring cells that still belong to the same region, and continue until the region is fully explored.

The attached code does not yet implement a full flood-fill breakup system, but it already prepares for this style of logic by storing densities in a sparse map and by marking localized regions dirty. That structure is exactly what a later connectivity pass would need.

7 Chunks and Incremental Mesh Regeneration

A major challenge is performance. Rebuilding an entire world after every punch would be too expensive. The usual solution is to divide the world into chunks and rebuild only the chunks affected by a change.

Mathematically, the world can be viewed as a union of chunks:

$$[\text{World} = \bigcup_i \text{Chunk}_i]$$

When terrain changes, we modify only the nearby samples, mark the relevant chunks as dirty, and rebuild those chunks later.

The implementation follows this architecture closely:

```
const CHUNK = 12;
const MAX_BUILD_PER_FRAME = 2;
const densityMap = new Map();
const chunks = new Map();
const dirtyQueue = [];

function getChunk(cx,cy,cz,create=true){
const k = chunkKey(cx,cy,cz);
let c = chunks.get(k);
if(!c && create){
c = {cx,cy,cz, dirty:true, vb:null, nb:null, count:0, visible:false, lastBuilt:0};
chunks.set(k, c);
dirtyQueue.push(c);
}
return c;
}

function markChunkDirty(cx,cy,cz){
const c = getChunk(cx,cy,cz,true);
if(!c.dirty){ c.dirty = true; dirtyQueue.push(c); }
}
```

This is exactly the kind of bookkeeping a real-time deformable terrain system needs. Chunking keeps the world manageable, dirty flags prevent wasted work, and the build queue lets the engine spread mesh updates across multiple frames.

The regeneration step then processes a limited number of dirty chunks per frame:

```
function processDirtyChunks(){
let built = 0;
const start = performance.now();
while(dirtyQueue.length && built < MAX_BUILD_PER_FRAME && performance.now() - start < 9){
const c = dirtyQueue.shift();
if(!c || !c.dirty) continue;
buildChunkMesh(c);
built++;
}
}
```

This is an important real-time optimization. Instead of freezing the game when terrain changes, the code rebuilds a small amount of geometry each frame. The player sees destruction immediately, but the expensive work is amortized over time.

8 Building the Mesh

The actual mesh generation walks through each chunk cell, samples the scalar field at the corners, subdivides the cube into tetrahedra, and builds triangles wherever the field changes sign.

The core of that process appears in the mesh builder:

```
const cubeVerts = [
[0,0,0],[1,0,0],[1,1,0],[0,1,0],
[0,0,1],[1,0,1],[1,1,1],[0,1,1]
];
const tets = [
[0,5,1,6],
[0,1,2,6],
[0,2,3,6],
[0,3,7,6],
[0,7,4,6],
[0,4,5,6]
];
```

By splitting the cube into tetrahedra, the algorithm can handle the surface more robustly and avoid many of the ambiguous cases that appear in naive cube triangulation.

Each generated triangle is then oriented using the gradient so that the normals point the correct way. That makes the lighting stable and prevents some faces from appearing inverted.

9 Collision with a Deformable World

Geometry alone is not enough. The player must also collide with the terrain. In a deformable world, collision cannot rely on a fixed mesh. It must query the same density field that the renderer uses.

The attached code handles this by sampling the density around the player's capsule and pushing the player out of solid regions along the local gradient.

```
function resolvePlayerCollision(){
player.grounded = false;
```

```

const samples = getCapsuleSamples(player.pos);
let pushX=0,pushY=0,pushZ=0, count=0;
for(const s of samples){
const d = sampleDensity(s[0], s[1], s[2]);
if(d < player.radius * 0.75){
const g = gradientAt(s[0], s[1], s[2]);
const penetration = (player.radius * 0.75) - d;
pushX += g[0] * penetration;
pushY += g[1] * penetration;
pushZ += g[2] * penetration;
count++;
if(g[1] > 0.35) player.grounded = true;
}
}
if(count){
player.pos[0] += pushX / count;
player.pos[1] += pushY / count;
player.pos[2] += pushZ / count;
if(pushY/count > 0) player.vel[1] = Math.max(player.vel[1], 0);
}
}

```

This is a very elegant trick. The collision system is not separate from the terrain system; it is the terrain system. Since both rendering and collision use the same field, the world remains consistent even after it is deformed.

10 Grabbing and Releasing Terrain

The attached implementation goes beyond simple destruction. It also lets the player grab a local region of terrain, detach it, carry it, throw it, and place it back.

That is an excellent demonstration of how editable scalar fields can support gameplay beyond breaking surfaces. A local block of density values is copied, the source region is carved out, and a debris mesh is built from the saved data.

```

function copyLocalRegion(center, size){
const half = Math.floor(size/2);
const data = [];
for(let x=-half; x<=half; x++) for(let y=-half; y<=half; y++) for(let z=-half; z<=half; z++){
const gx = Math.round(center[0]) + x;
const gy = Math.round(center[1]) + y;
const gz = Math.round(center[2]) + z;
data.push({x:x,y:y,z:z,v:getDensityPoint(gx,gy,gz)});
}
return data;
}

function carveLocalRegion(center, size, amount){
const half = Math.floor(size/2);
for(let x=-half; x<=half; x++) for(let y=-half; y<=half; y++) for(let z=-half; z<=half; z++){
const gx = Math.round(center[0]) + x;
const gy = Math.round(center[1]) + y;
const gz = Math.round(center[2]) + z;
const d = Math.hypot(x,y,z);
if(d > half + 0.4) continue;
const t = 1 - d / (half + 0.4);

```

```
const fall = t*t*(3-2*t);
addDensityPoint(gx,gy,gz, amount * fall * fall);
markPointDirty(gx,gy,gz);
}
}
```

This is conceptually similar to grabbing a chunk of clay from a sculpture. The region is not merely animated; it is physically removed from the field and later reinserted.

11 The Complete Pipeline

The whole system can be summarized as a pipeline:

Player input ↓ Density field modification ↓ Dirty chunk marking ↓ Surface extraction ↓
Buffer regeneration ↓ Rendering and collision

The key point is that each stage depends on the previous one, but each stage is also local. A punch does not require a full world rebuild; only the region around the impact changes. That locality is what makes the system fast enough for real-time gameplay.

12 Conclusion

Modern destructible terrain systems are not just visual tricks. They combine voxel-style storage, scalar fields, interpolation, graph reasoning, chunked mesh regeneration, and field-based collision into a single coherent architecture.

The result is a world that can be sculpted naturally while still remaining playable and efficient. The attached JavaScript demonstrates that this is not only a theoretical idea: it is a practical pipeline. Densities are sampled on demand, terrain is deformed through smooth falloff functions, chunks are rebuilt incrementally, and the player can both collide with and manipulate the same surface.

Once these pieces are understood together, the impressive destruction seen in modern games stops feeling magical and starts looking like a carefully designed mathematical system.

A Prerequisites

A.1 Vectors

Understand:

- vector addition
- dot product
- cross product
- normalization

These are needed for gradients, normals, and camera control.

A.2 Linear Algebra

Useful concepts:

- matrices
- coordinate transformations
- projection
- camera systems

The rendering code uses model, view, and projection matrices to place the terrain and the player in 3D space.

A.3 Graph Theory

Essential topics:

- BFS
- DFS
- connected components
- flood fill

These are the tools behind detached chunks, cave detection, and region extraction.

A.4 Interpolation

Surface extraction relies heavily on interpolation.

Important examples include:

- linear interpolation
- bilinear interpolation
- trilinear interpolation

Without interpolation, the terrain would appear blocky rather than smooth.